

Лабораторная работа №8

Основы информационной безопасности

Павлова Татьяна Юрьевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
5	Ответы на контрольные вопросы	14
6	Выводы	15

Список иллюстраций

4.1	Шифрование двух текстов ч.1	9
4.2	Шифрование двух текстов ч.2	10
4.3	Шифрование двух текстов ч.3	10
4.4	Результат работы программы	11

Список таблиц

1 Цель работы

Целью данной работы является освоение на практике применения режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом

2 Задание

Два текста кодируются одним ключом (однократное гаммирование). Требуется, не зная ключа и не стремясь его определить, прочитать оба текста. Необходимо разработать приложение, позволяющее шифровать и дешифровать тексты P_1 и P_2 в режиме однократного гаммирования. Приложение должно определить вид шифротекстов C_1 и C_2 обоих текстов P_1 и P_2 при известном ключе; Необходимо определить и выразить аналитически способ, при котором злоумышленник может прочитать оба текста, не зная ключа и не стремясь его определить.

3 Теоретическое введение

Исходные данные.

Две телеграммы Центра:

P_1 = НаВашиходящийот1204

P_2 = ВСеверныйфилиалБанка

Ключ Центра длиной 20 байт: $K = 05\ 0C\ 17\ 7F\ 0E\ 4E\ 37\ D2\ 94\ 10\ 09\ 2E\ 22\ 57\ FF\ C8\ 0B\ B2\ 70\ 54$

Шифротексты обеих телеграмм можно получить по формулам режима однократного гаммирования:

$$C_1 = P_1 \oplus K,$$

$$C_2 = P_2 \oplus K. \quad (8.1)$$

Открытый текст можно найти, зная шифротекст двух телеграмм, зашифрованных одним ключом. Для это оба равенства (8.1) складываются по модулю 2. Тогда с учётом свойства операции XOR

$$1 \oplus 1 = 0, 1 \oplus 0 = 1 \quad (8.2)$$

получаем:

$$C_1 \oplus C_2 = P_1 \oplus K \oplus P_2 \oplus K = P_1 \oplus P_2.$$

Предположим, что одна из телеграмм является шаблоном — т.е. имеет текст фиксированный формат, в который вписываются значения полей. Допустим, что злоумышленнику этот формат известен. Тогда он получает достаточно много пар

$C_1 \oplus C_2$ (известен вид обеих шифровок). Тогда зная P_1 и учитывая (8.2), имеем:

$$C_1 \oplus C_2 \oplus P_1 = P_1 \oplus P_2 \oplus P_1 = P_2. (8.3)$$

Таким образом, злоумышленник получает возможность определить те символы сообщения P_2 , которые находятся на позициях известного шаблона сообщения P_1 . В соответствии с логикой сообщения P_2 , злоумышленник имеет реальный шанс узнать ещё некоторое количество символов сообщения P_2 . Затем вновь используется (8.3) с подстановкой вместо P_1 полученных на предыдущем шаге новых символов сообщения P_2 . И так далее. Действуя подобным образом, злоумышленник даже если не прочитает оба сообщения, то значительно уменьшит пространство их поиска.

[course]

4 Выполнение лабораторной работы

Я выполняла лабораторную работу на языке программирования Python, используя функции, реализованные в лабораторной работе №7.

Используя функцию для генерации ключа, генерирую ключ, затем шифрую два разных текста одним и тем же ключом (рис. [fig:001]). (рис. [fig:002]).(рис. [fig:003]).

```
import random

def text_to_codes(text):
    """Преобразует строку в список кодов символов"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes

def codes_to_text(codes):
    """Преобразует список кодов обратно в строку"""
    text = ""
    for i in range(len(codes)):
        text += chr(codes[i])
    return text
```

Рисунок 4.1: Шифрование двух текстов ч.1

```
def xor(b1, b2):
    """XOR двух списков кодов"""
    if len(b1) != len(b2):
        print("Ошибка: длины не соответствуют")
        return None
    result = []
    for i in range(len(b1)):
        new = b1[i]^b2[i]
        result.append(new)
    return result

def generate_key(length):
    """Генерирует случайный ключ заданной длины"""
    key = []
    for i in range(length):
        key.append( random.randint(0, 255))
    return key
```

Рисунок 4.2: Шифрование двух текстов ч.2

```
# основная программа
plain1 = ("С Новым Годом, друзья!")
plain2 = ("У Слона домом, огоро!!")

# генерация ключа
key = generate_key(len(plain1))

# преобразование ключа в коды
plain_codes1 = text_to_codes(plain1)
plain_codes2 = text_to_codes(plain2)

# шифрование
cipher_bytes1 = xor(plain_codes1, key)
cipher_text1 = codes_to_text(cipher_bytes1)

cipher_bytes2 = xor(plain_codes2, key)
cipher_text2 = codes_to_text(cipher_bytes2)

# дешифровка
decrypted_bytes1 = xor(cipher_bytes1, key)
decrypted_text1 = codes_to_text(decrypted_bytes1)

decrypted_bytes2 = xor(cipher_bytes2, key)
decrypted_text2 = codes_to_text(decrypted_bytes2)

print('Открытый текст: ', plain1, "\nКлюч: ", key, "\nШифротекст: ", cipher_text1, "\nИсходный текст: ", decrypted_text1)
print('Открытый текст: ', plain2, "\nКлюч: ", key, "\nШифротекст: ", cipher_text2, "\nИсходный текст: ", decrypted_text2)

r = xor(cipher_bytes1, cipher_bytes2)
print('Расшифровать второй текст, зная первый: ', codes_to_text(xor(plain_codes1, r)))
print('Расшифровать первый текст, зная второй: ', codes_to_text(xor(plain_codes2, r)))
```

Рисунок 4.3: Шифрование двух текстов ч.3

Расшифровываю оба текста сначала с помощью одного ключа, затем предполагаю, что мне неизвестен ключ, но известен один из текстов и уже расшифровываю второй, зная шифротексты и первый текст (рис. [fig:004]).

```

[tanya@user lab08]$ python3 lab2.py
Открытый текст: С Новым Годом, друзья!
Ключ: [79, 233, 68, 187, 22, 160, 178, 64, 120, 25, 64, 124, 234, 118, 137, 252, 170, 69, 223, 208, 17, 146]
Шифротекст: 3ЕЉФӨР'жЧтЕZоңІӨкү³
Исходный текст: С Новым Годом, друзья!
Открытый текст: У Слона домов, оного!!
Ключ: [79, 233, 68, 187, 22, 160, 178, 64, 120, 25, 64, 124, 234, 118, 137, 252, 170, 69, 223, 208, 17, 146]
Шифротекст: ӱӱеШӱ*³ьЧЗәZежзәӱ0³
Исходный текст: У Слона домов, оного!!
Расшифровать второй текст, зная первый: У Слона домов, оного!!
Расшифровать первый текст, зная второй: С Новым Годом, друзья!

```

Рисунок 4.4: Результат работы программы

Листинг программы

```

import random

def text_to_codes(text):
    """XXXXXXXXXXXXXXXX XXXXXXX XXXXXXX XXXXXXX XXXXXXX XXXXXXX"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes

def codes_to_text(codes):
    """XXXXXXXXXXXXXXXX XXXXXXX XXXXXXX XXXXXXX XXX XXXXXXX"""
    text = ""
    for i in range(len(codes)):
        text += chr(codes[i])
    return text

def xor(b1, b2):
    """XOR XXXXX XXXXXXX XXXXXXX"""

```

```

if len(b1) != len(b2):
    print("Error: Invalid key length")
    return None

result = []
for i in range(len(b1)):
    new = b1[i]^b2[i]
    result.append(new)
return result

def generate_key(length):
    """Generate a key of length 16"""
    key = []
    for i in range(length):
        key.append( random.randint(0, 255))
    return key

# Example usage
plain1 = ("HELLO WORLD, HELLO!")
plain2 = ("HELLO WORLD, HELLO!!")

# Generate key
key = generate_key(len(plain1))

# Encrypt
plain_codes1 = text_to_codes(plain1)
plain_codes2 = text_to_codes(plain2)

```

```

# 000000000000
cipher_bytes1 = xor(plain_codes1, key)
cipher_text1 = codes_to_text(cipher_bytes1)

cipher_bytes2 = xor(plain_codes2, key)
cipher_text2 = codes_to_text(cipher_bytes2)

# 000000000000
decrypted_bytes1 = xor(cipher_bytes1, key)
decrypted_text1 = codes_to_text(decrypted_bytes1)

decrypted_bytes2 = xor(cipher_bytes2, key)
decrypted_text2 = codes_to_text(decrypted_bytes2)

print('00000000 0000: ', plain1, "\n0000: ", key, '\n0000000000: ', cipher_
print('00000000 0000: ', plain2, "\n0000: ", key, '\n0000000000: ', cipher_

r = xor(cipher_bytes1, cipher_bytes2)
print('000000000000 00000 0000, 0000 00000: ', codes_to_text(xor(plain_codes1, cipher_bytes2)),
print('000000000000 00000 0000, 0000 00000: ', codes_to_text(xor(plain_codes2, cipher_bytes2)))

```

5 Ответы на контрольные вопросы

1. Как, зная один из текстов (P_1 или P_2), определить другой, не зная при этом ключа? - Для определения другого текста (P_2) можно просто взять зашифрованные тексты $C_1 \oplus C_2$, далее применить XOR к ним и к известному тексту: $C_1 \oplus C_2 \oplus P_1 = P_2$.
2. Что будет при повторном использовании ключа при шифровании текста? - При повторном использовании ключа мы получим дешифрованный текст.
3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов? - Режим шифрования однократного гаммирования одним ключом двух открытых текстов осуществляется путем XOR-ирования каждого бита первого текста с соответствующим битом ключа или второго текста.
4. Перечислите недостатки шифрования одним ключом двух открытых текстов - Недостатки шифрования одним ключом двух открытых текстов включают возможность раскрытия ключа или текстов при известном открытом тексте.
5. Перечислите преимущества шифрования одним ключом двух открытых текстов - Преимущества шифрования одним ключом двух открытых текстов включают использование одного ключа для зашифрования нескольких сообщений без необходимости создания нового ключа и выделения на него памяти.

6 Выводы

В ходе выполнения лабораторной работы, я освоила на практике применения режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом